

A Physics-Informed Neural Network for Option Pricing under Rough Volatility: Meshfree Solution of the Markovian-Lifted PDE and Fast Parametric Calibration

[Authors / affiliations]

Complete draft (IMRAD). July 6, 2026

Abstract

Rough volatility models fit equity and cryptocurrency option markets well but are non-Markovian, so they admit no finite-dimensional pricing partial differential equation (PDE). The Markovian lift recovers a PDE by approximating the fractional kernel with n exponential factors, at the cost of raising the PDE dimension to $n+1$, which makes the problem a natural testbed for mesh-free neural solvers. We solve the lifted rough-Heston PDE with a physics-informed neural network (PINN) that minimizes the strong-form residual, using a hard-constrained initial condition, asymptotic boundary losses, self-adaptive loss weighting, a maturity curriculum, and Monte-Carlo-informed factor sampling. We validate by decomposing the error, in price space where the signed errors add exactly, into a *solve-error* (the network versus an independent lifted Monte Carlo at the same n) and a *lift-error* (lifted Monte Carlo versus the true model). Our main methodological result is that the solve-error grows only sub-linearly in the lifted dimension — staying below about eleven price basis points up to $n=32$ over three seeds — whereas a naive full-tensor finite-difference grid grows as 50^{n+1} and is already infeasible by $n=8$; we are explicit that structured grid schemes fare better but remain impractical for this PDE. The solve/lift crossover yields a concrete rule for choosing n , which shifts to larger n at longer maturities. Calibrated to real Bitcoin options, the model returns a rough Hurst exponent (about 0.09) with strong negative correlation. Finally, a parametric PINN that takes the five model parameters as inputs — holding the spot variance and maturity fixed — enables gradient-based recalibration; although the network alone is too inaccurate for production calibration, using it to warm-start a short local optimization against the true pricer reaches from-scratch fit quality at roughly nine times fewer true-model evaluations. Mesh-free PINNs thus scale gracefully in the lifted dimension where grids cannot, and are most valuable as a differentiable warm-start for calibration.

Keywords: physics-informed neural networks; rough volatility; option pricing; Markovian lift; curse of dimensionality; model calibration; scientific machine learning.

1 Introduction

Pricing a derivative under a stochastic model amounts to solving a partial differential equation (PDE). For most models this PDE is low-dimensional and classical grid-based solvers are effective. An important and empirically well-supported class of models, however, breaks this picture: under *rough volatility* the instantaneous variance is driven by a fractional noise with Hurst exponent

$H < 1/2$, so the variance process is neither Markovian nor a semimartingale and there is *no* finite-dimensional pricing PDE (Gatheral et al., 2018; Bennedsen et al., 2022; El Euch and Rosenbaum, 2019). The standard remedy is the *Markovian lift*: the fractional kernel is approximated by a sum of n exponentials, restoring a PDE at the cost of raising its dimension to $n + 1$ (Abi Jaber and El Euch, 2019; Abi Jaber, 2019). Because n directly sets the dimension of the PDE while controlling a known approximation error, the lifted rough-volatility problem is a clean, economically motivated testbed for mesh-free neural PDE solvers and for studying the curse of dimensionality.

Physics-informed neural networks (PINNs) solve a PDE by minimizing its residual at sampled collocation points, using automatic differentiation and requiring no labelled solution data (Raissi et al., 2019). Being mesh-free, they sidestep the exponential growth in grid points that makes finite differences infeasible in high dimensions — the same motivation behind deep PDE solvers such as the deep Galerkin method (Sirignano and Spiliopoulos, 2018) and deep BSDE solvers (Han et al., 2018). Two properties make the approach attractive for derivatives: the physics (the PDE) is known exactly, so market data enter only through the model coefficients; and, once trained across a parameter range, the network is differentiable in those parameters, enabling gradient-based calibration.

On the finance side, and building on long-memory continuous-time volatility (Comte and Renault, 1998), the rough-volatility paradigm reproduces the observed roughness of realized variance and the term structure of at-the-money skew far better than classical stochastic-volatility models such as Heston (1993) (Gatheral et al., 2018; Bennedsen et al., 2022; Bayer et al., 2016), and has microstructural foundations in high-frequency trading (El Euch et al., 2018); the rough Heston model in particular admits a semi-analytic characteristic function (El Euch and Rosenbaum, 2019). Calibration, however, remains computationally demanding, which motivates fast neural surrogates. We use Bitcoin (BTC) options as a testbed: the market is liquid, exhibits a pronounced negative skew, and calibrates to a rough Hurst exponent.

Our contributions, stated conservatively, are the following.

1. **A dimension-scaling result (primary, methodological).** The PINN’s *solve-error* — how well it solves the PDE it is given — grows only *sub-linearly* in the lifted dimension, staying below about 11 price bp up to $n = 32$, while a naive full-tensor finite-difference grid blows up exponentially and is already infeasible by $n = 8$. We are explicit that $50^{(n+1)}$ is the *naive* grid; structured solvers do better but degrade with n and are not standard for this PDE, so the honest claim is that finite differences are *impractical*, not impossible.
2. **An error decomposition and a rule for choosing n .** We separate the *solve-error* (network versus lifted Monte Carlo at the same n) from the *lift-error* (lifted Monte Carlo versus the true model), report them in price space where the signed errors add exactly, and use their crossover to choose n ; we show how the crossover shifts with maturity.
3. **Empirical calibration.** We calibrate rough Heston to real Deribit BTC options and obtain a rough Hurst exponent with strong negative correlation.
4. **Fast parametric recalibration (applied).** A parametric PINN plus a *hybrid* (PINN warm-start followed by a few local Fourier polish steps) reaches from-scratch calibration quality at roughly nine times fewer Fourier evaluations; we report the parametric network’s standalone accuracy gap honestly.

We emphasize what we do *not* claim. We are not the first to solve the lifted (rough) Heston PDE with a mesh-free neural method: Papantoleon and Rou (2025) already do so. Our contribution is

a specific combination — a PINN residual formulation, the solve/lift decomposition and n -crossover analysis, real crypto calibration, and the parametric hybrid — detailed next.

2 Related work

Mesh-free deep pricing on the lifted PDE (closest prior art). Papapantoleon and Rou (2025) price European options in (rough) diffusion models by reformulating the pricing PDE as an energy-minimization problem and approximating it in a time-stepping fashion with deep networks (a “deep gradient flow”), demonstrated specifically on the lifted Heston model and respecting the large-moneyness asymptotics and known price bounds. This is the work closest to ours in both target (the lifted rough-vol PDE) and spirit (mesh-free deep pricing). It differs in method: theirs is a time-stepping energy/variational scheme, whereas ours is a PINN that directly minimizes the strong-form PDE residual with a hard-constrained initial condition. It also differs in scope: we add a quantitative n -scaling and solve/lift decomposition, calibration to real market data, and a parametric recalibration hybrid, none of which is the focus there.

Other neural solvers for rough volatility. Jacquier and Zurič (2023) solve the path-dependent PDEs of rough volatility by interpreting them as backward stochastic differential equations and using a reservoir-type *random* neural network, which reduces training to a least-squares regression with convergence guarantees. This shares our target class but uses a fundamentally different mechanism (BSDE plus random features, not residual minimization) and does not study dimension scaling or calibration speed, which are central here.

Neural calibration surrogates (the calibration foil). The dominant paradigm for fast (rough-)volatility calibration is the supervised surrogate of Bayer et al. (2019) and Horvath et al. (2021): a neural network is trained offline on a large grid of precomputed model prices or implied volatilities and then used as a fast pricing map inside a calibration loop. This is purely data-driven and carries no PDE consistency — the network never sees the governing equation. Our parametric network is instead *physics-informed*: it learns from the PDE residual rather than from precomputed labels. This distinction is what makes our hybrid meaningful — the physics-trained network supplies a warm start that a few true-model evaluations then refine (Section 4.4).

Foundations. We build on PINNs (Raissi et al., 2019) and, more broadly, mesh-free deep solvers for high-dimensional PDEs (Sirignano and Spiliopoulos, 2018; Han et al., 2018); on rough volatility (Gatheral et al., 2018; El Euch and Rosenbaum, 2019) and its multifactor Markovian lift (Abi Jaber and El Euch, 2019); and on classical transform pricing (Heston, 1993; Carr and Madan, 1999) for our independent ground truth. Two PINN training techniques we adopt are self-adaptive loss weighting by homoscedastic uncertainty (Kendall et al., 2018) and causal (curriculum) training in the time variable (Wang et al., 2024).

3 Methods

3.1 Rough Heston and the Markovian lift

Under rough Heston the variance is driven by the fractional kernel $K(t) = t^{\alpha-1}/\Gamma(\alpha)$ with $\alpha = H + \frac{1}{2}$ and $H < \frac{1}{2}$, which is non-Markovian (Gatheral et al., 2018; El Euch and Rosenbaum, 2019). The Markovian lift (Abi Jaber and El Euch, 2019; Abi Jaber, 2019) approximates $K(t) \approx$

$\sum_{i=1}^n c_i e^{-x_i t}$, introducing n Ornstein–Uhlenbeck factors U^i with $V = V_0 + \sum_i c_i U^i$. The discounted price $P(t, x, u)$, with $x = \log S$, then solves the $(n+1)$ -dimensional backward Kolmogorov PDE

$$\begin{aligned} 0 = & P_t + \left(r - \frac{1}{2}V\right)P_x + \frac{1}{2}V P_{xx} + \sum_i \left[-x_i u_i + \lambda(\theta - V)\right]P_{u_i} \\ & + \frac{1}{2}\nu^2 V \sum_{i,j} P_{u_i u_j} + \rho\nu V \sum_i P_{xu_i} - rP, \end{aligned} \quad (1)$$

with terminal condition $P(T, x, u) = \text{payoff}(e^x)$ and $V = V_0 + \sum_i c_i u_i$. All factors share one Brownian motion, hence the full double sum in the u -diffusion and the cross term with x . The parameter n is thus a tunable knob on the PDE dimension. We build (c_i, x_i) by the geometric-quadrature rule of Abi Jaber and El Euch (2019); the relative L^2 kernel error decreases monotonically from 25.6% at $n = 5$ to 0.39% at $n = 100$.

3.2 Two independent ground truths

We validate against two mutually independent pricers. (i) A semi-analytic *Fourier* pricer solves the fractional Riccati equation for the rough Heston characteristic function (El Euch and Rosenbaum, 2019) and inverts it by Fourier integration (Carr and Madan, 1999); it does not use the lift, so it is the ground truth of the true model ($n \rightarrow \infty$). In the flat-volatility limit it matches Black–Scholes to 3.7×10^{-7} . (ii) A *lifted Monte Carlo* simulator integrates the n -factor lifted SDE with an exponential-integrator Euler scheme that treats the stiff $-x_i U^i$ term exactly — the mean reversions span roughly ten orders of magnitude for small H , so explicit schemes diverge. It matches Black–Scholes within Monte Carlo error. Because the two pricers approximate different objects (an ODE versus the model dynamics), their agreement is a genuine cross-check. Crucially, *at finite n the lifted MC is the ground truth of the PDE the network solves*, whereas Fourier is the ground truth of the model; this underlies the decomposition in Section 3.4.

3.3 Physics-informed neural network

We solve (1) with a PINN (Raissi et al., 2019), a mesh-free residual method in the spirit of Sirignano and Spiliopoulos (2018); Han et al. (2018). The network is a tanh multilayer perceptron of (τ, x, u) with $\tau = T - t$, so the terminal condition becomes an initial condition at $\tau = 0$. We hard-wire that condition through the ansatz

$$P(\tau, x, u) = P_{\text{base}}(\tau, x) + \tau N_\theta(\tau, x, u), \quad (2)$$

whose τ prefactor forces the learned correction N_θ to vanish at $\tau = 0$. The baseline P_{base} is a Black–Scholes price anchored not at the spot variance V_0 but at the model’s expected *integrated* variance, $w(\tau) = \int_0^\tau \mathbb{E}[V_s] ds$, which captures mean reversion of V from V_0 toward θ ; anchoring at V_0 leaves a systematic level bias that the network would have to undo.

The loss combines the interior residual of (1) with asymptotic boundary losses $P \rightarrow 0$ as $S \rightarrow 0$ and $P \rightarrow S - Ke^{-r\tau}$ as $S \rightarrow \infty$, resampled each iteration. PINNs are prone to optimization pathologies — loss terms on very different scales, and violations of temporal causality that can steer training toward wrong solutions (Krishnapriyan et al., 2021; Wang et al., 2022) — and two stabilizers address these here. First, the residual and the two boundary terms are combined with self-adaptive per-term weights, using the learnable homoscedastic-uncertainty scheme of Kendall et al. (2018) in the self-adaptive spirit of McClenny and Braga-Neto (2023), which removes hand-tuned weights. Second, a τ -curriculum trains short maturities first and expands the horizon,

respecting temporal causality (Wang et al., 2024). Finally, factor collocation is informed by a short pre-flight Monte Carlo: rather than sampling u in an arbitrary symmetric box, we sample each factor from the mean and standard deviation it actually attains under the lifted dynamics, which is essential for reproducing the skew.

3.4 Validation protocol and error decomposition

The network is trained at fixed strike $K = 1$ and prices as a function of spot; the fixed-spot smile $C(1, K)$ is recovered by the call’s degree-one homogeneity,

$$C(1, K) = K C(1/K, 1), \quad (3)$$

which holds because the variance dynamics are independent of the spot level. Equation (3) is exact, not an approximation, and is applied consistently in every comparison in this paper. To show why it is essential we ran a protocol ablation: omitting the rescaling and comparing the mismatched options directly inflates the apparent error to ~ 2700 vol bp. That figure is a property of the flawed comparison, not a residual error of our method, whose accuracy is always the decomposed quantity defined next. We never report a lone “PINN-versus-Fourier” number; instead we decompose, per strike and in price space where the signed errors add exactly,

$$\underbrace{(\text{PINN} - \text{Fourier})}_{\text{total}} = \underbrace{(\text{PINN} - \text{MC})}_{\text{solve-error}} + \underbrace{(\text{MC} - \text{Fourier})}_{\text{lift-error}}. \quad (4)$$

The solve-error is what training, architecture, and sampling can improve; the lift-error is a model approximation independent of the network, shrinking only with n . We report price RMSE in basis points of spot (where (4) is additive) and give implied-volatility equivalents where useful; the two components have opposite signs and so do not add in RMS.

3.5 Parametric PINN

For instant recalibration we train one network that also takes the five model parameters $(H, \nu, \rho, \lambda, \theta)$ as inputs, over a box around the calibrated BTC values ($H \in [0.04, 0.20]$, $\nu \in [0.10, 0.60]$, $\rho \in [-0.95, -0.20]$, $\lambda \in [0.5, 4.0]$, $\theta \in [0.08, 0.35]$); V_0 and the maturity are fixed. Since the lift weights depend on H , they are precomputed on an H -grid and gathered per collocation point, while the parameters enter the network as exact continuous inputs. The number of factors is fixed by the crossover rule of Section 4.1 ($n = 10$). The network is differentiable in the model parameters, which enables gradient calibration.

3.6 Calibration procedures and data

We calibrate three ways: (a) *Fourier-in-the-loop*, differential evolution with the Fourier pricer inside the objective; (b) *PINN gradient*, Adam on the parameters using the parametric network as a differentiable pricer, with zero Fourier calls; and (c) *hybrid*, the PINN’s single-shot solution as an initial guess refined by a few local Fourier polish steps (Nelder–Mead). The objective is the vega-weighted implied-volatility RMSE against the market smile. The market data are end-of-day BTC option quotes from Deribit, a cryptocurrency derivatives exchange and data provider, retrieved through its public interface; we set $r = 0$ and infer the forward from the quotes, following the crypto convention, and recompute implied volatilities from mid prices with a robust Black–Scholes inverter.

Table 1: Dimension scaling (canonical BTC calibration, $T = 0.15$; solve-error is 3-seed mean $\pm$ SD; price RMSE in basis points of spot).

n	solve-error (px bp)	lift-error (px bp)	train time (s)	FD grid $50^{(n+1)}$
4	5.4 ± 1.0	16.4	397	3.1×10^8
8	6.1 ± 1.4	7.9	386	2.0×10^{15}
16	8.8 ± 1.6	3.5	591	7.6×10^{28}
32	10.7 ± 2.0	1.7	457	1.2×10^{56}

4 Results

4.1 Dimension scaling: sub-linear solve-error versus FD blow-up

Training the same architecture (width 64, depth 4) at $n = 4, 8, 16, 32$ on the canonical BTC calibration at $T = 0.15$ and measuring the solve-error against the lifted MC over three seeds (Table 1, Figure 1), the solve-error grows only mildly — from 5.4 ± 1.0 to 10.7 ± 2.0 price bp, about a factor of two over an eight-fold increase in dimension. It is not flat (an earlier single-seed run misleadingly suggested so), but grows far more slowly than the dimension. With only three seeds the error bars at adjacent n overlap, so we read the trend across the full range rather than claiming significance between neighbouring points; the roughly two-fold growth from $n = 4$ to $n = 32$ is the robust statement. Wall-clock cost to the plateau stays bounded (about 6–10 minutes on a CPU; per-iteration cost roughly constant at 55–62 ms) because the factor Hessian is vectorized.

By contrast, a naive full-tensor finite-difference grid on the same $(n+1)$ -dimensional PDE at 50 nodes per dimension needs $50^{(n+1)}$ points: 3.1×10^8 at $n = 4$, 2.0×10^{15} at $n = 8$, and 1.2×10^{56} at $n = 32$. Even $n = 8$ (about 16 petabytes in double precision) is infeasible, while the network trains every n on a laptop CPU. We state the contrast honestly: $50^{(n+1)}$ is the *naive* grid. Structured schemes — sparse grids, ADI, low-rank tensor-train — push the tractable dimension higher, but their cost and accuracy still degrade with n (sparse-grid node counts grow super-linearly; tensor-train needs low-rank structure that the coupled ρ - ν cross terms erode), and none is standard or validated for the lifted rough-volatility PDE. The honest claim is that a competitive finite-difference solve at $n = 16$ – 32 is *impractical*, not provably impossible. Gentle solve-error growth with bounded cost, against this blow-up, is the curse-of-dimensionality argument for the mesh-free approach.

4.2 The solve/lift decomposition and a crossover rule for n

The decomposition explains the record. A single “PINN vs Fourier” implied-vol RMSE of ~ 114 – 127 vol bp at $n = 4$ would read as network error; split by (4), it is dominated by the lift. In the full configuration the $n = 4$ error is 37 vol bp of solve-error plus 133 of lift (partially cancelling to a total of 121), and in the correlation-free case it is only 8 of solve-error on top of 133 of lift. The network already solves its PDE to well under 50 vol bp; the apparent gap was model approximation.

Because the lift-error shrinks monotonically (133, 63, 27, 12 vol bp at $n = 4, 8, 16, 32$; equivalently $16.4 \rightarrow 1.7$ price bp) while the solve-error grows mildly, the two cross. At $T \approx 0.15$ the crossover sits near $n = 8$ – 10 (Figure 1): below it the model approximation dominates and one should add factors; above it the network’s own error dominates and factors are wasted. This gives a concrete rule for choosing n . The crossover also moves with maturity: over a T -sweep the solve-error stays bounded everywhere (roughly 1–9.5 price bp across $T \in \{0.05, 0.15, 0.5\}$ and $H \in \{0.05, 0.10, 0.15\}$), but the lift-error grows with T — from 8.6 to 31.8 price bp at $n = 4$ as T goes $0.05 \rightarrow 0.5$ — so at long

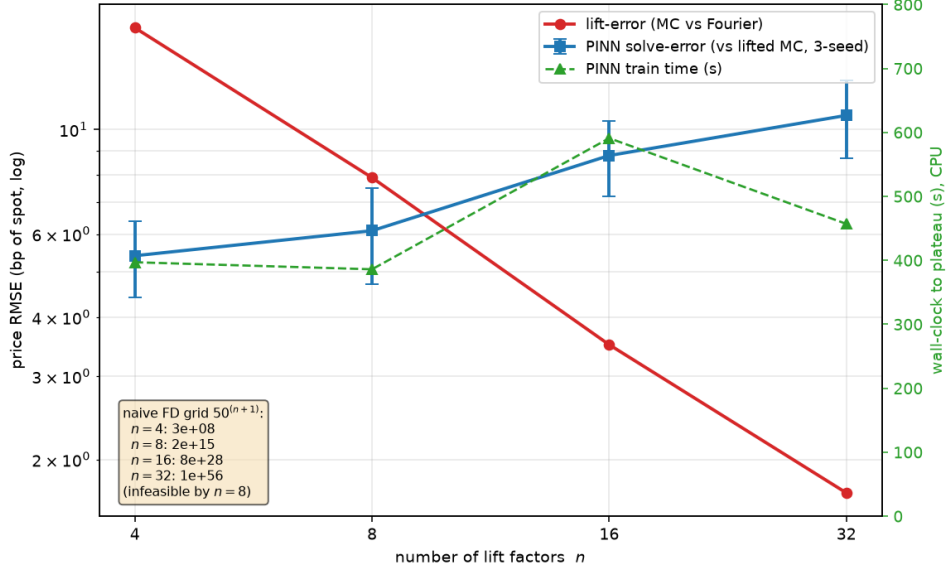


Figure 1: Dimension scaling. PINN solve-error (blue, 3-seed mean $\pm$ SD) grows sub-linearly and stays under ~ 11 pricebp to $n = 32$, while lift-error (red) falls monotonically and training time (green) stays bounded; the naive finite-difference grid (inset) is infeasible by $n = 8$.

maturities the lift still exceeds the solve floor at $n = 16$ and the crossover shifts to higher n . Longer horizons need more factors.

4.3 Calibration to real BTC options gives a rough Hurst exponent

Calibrated to the Deribit BTC surface with a full-budget optimizer (differential evolution, 60 generations plus local polish; all 13 maturities, up to 8 points per maturity, 97 points), the model yields $H = 0.090$, $\rho = -0.792$, $V_0 = 0.103$, $\theta = 0.253$, $\lambda = 2.567$, $\nu = 0.300$, at a vega-weighted implied-vol RMSE of 496 volbp (Figure 2). The Hurst exponent is well inside the rough regime and the correlation strongly negative, both economically sensible for crypto. We report the fit as deliberately coarse: crypto smiles are wide and the lifted rough-Heston form is stiff, and a single 496-bp figure is honest about that.

4.4 Parametric PINN and the calibration hybrid

The parametric network varies the five model parameters ($H, \nu, \rho, \lambda, \theta$) while holding the spot variance V_0 and the maturity T fixed; a single network covering this box is necessarily less accurate at any one point than a network trained there. At the box centre the parametric solve-error is 11.0 ± 4.7 pricebp against 5.9 ± 1.2 for a single-point network at the same $n = 10$ — about a two-fold coverage penalty. The penalty is uneven: it stays near 9–11 pricebp at typical points but rises to 47.7 in the rough low- H corner and 29.8 at high vol-of-vol, where the PDE is hardest. We report this honestly; a larger training budget would shrink the corner penalty.

This limits standalone PINN calibration but not its use as a warm start. Calibrating the BTC $T \approx 0.15$ slice three ways (Table 2), the Fourier-in-the-loop optimizer reaches 23.7 volbp using 2018 Fourier evaluations in about 168 s. PINN-only gradient descent is fast (about 2 s, zero Fourier calls) but poor — 222 ± 20 volbp under the true model — because it calibrates to its own

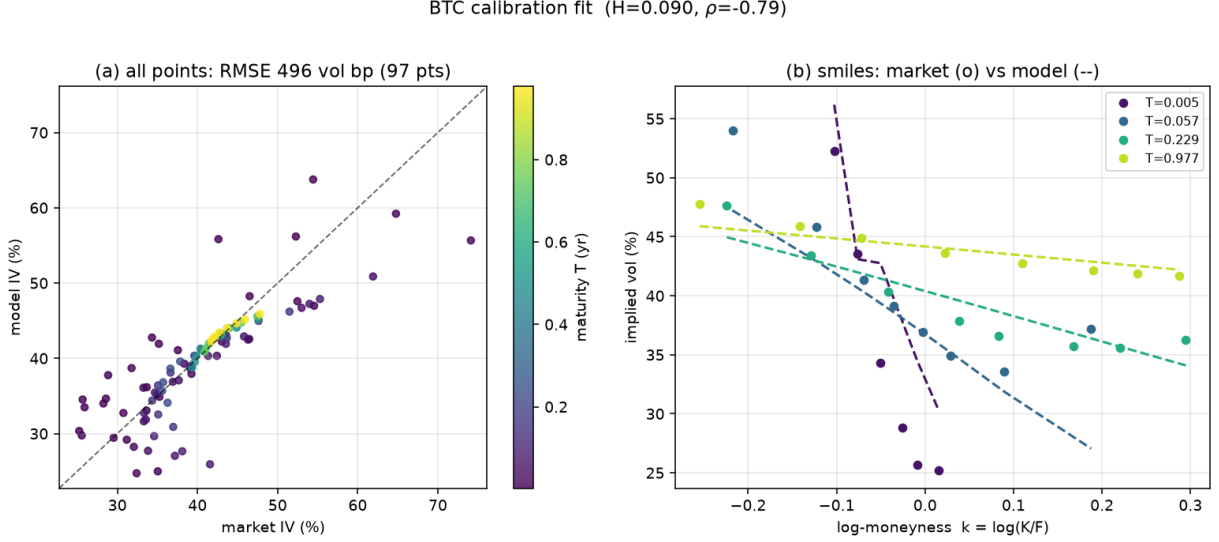


Figure 2: Rough Heston calibrated to the Deribit BTC surface (97 points, 13 maturities) at the canonical fit ($H = 0.090$, $\rho = -0.79$; RMSE 496 vol bp). (a) market versus model implied volatility at all points (dashed line: identity; colour = maturity). (b) per-maturity smiles for four representative maturities: market (markers) versus model (dashed). The fit is deliberately coarse: the residual spread reflects a known limitation of the lifted rough-Heston form on wide, richly curved crypto smiles — a model-expressiveness issue, not a solver artifact — and is orthogonal to the solve-error studied elsewhere.

Table 2: Calibration of the BTC $T \approx 0.15$ slice, three ways (3 PINN seeds; all judged by the true Fourier model over the same feasible region).

method	fit RMSE (vol bp)	wall-clock (s)	Fourier evals
Fourier-loop from scratch	23.7	168	2018
PINN-only (single-shot)	222 ± 20	~ 2	0
Hybrid (PINN + Fourier polish)	29.4 ± 7.1	23	237 ± 46

imperfect prices and, as an honest aside, because this single-maturity slice optimum lies outside the parametric training box (it wants $\nu \approx 0.87$, $\theta \approx 0.39$). The hybrid resolves this: seeding the local Fourier polish with the PINN’s guess reaches 29.4 ± 7.1 vol bp — comparable to from-scratch on a 34–52% smile — in only 237 ± 46 Fourier evaluations and about 23 s, roughly nine times fewer Fourier calls and seven times faster. The applied value of the parametric network is therefore as a calibration warm start, not a standalone pricer.

5 Discussion

Interpretation of the scaling result. The central finding is that the network’s own error in solving the lifted PDE degrades only gently as the PDE dimension grows, while grid-based solvers become impossible. We deliberately state this in its weak, honest form. The solve-error is not constant in n — a claim an early single-seed experiment appeared to support but which three-seed statistics refuted — it grows sub-linearly, roughly doubling from $n = 4$ to $n = 32$. Likewise, the

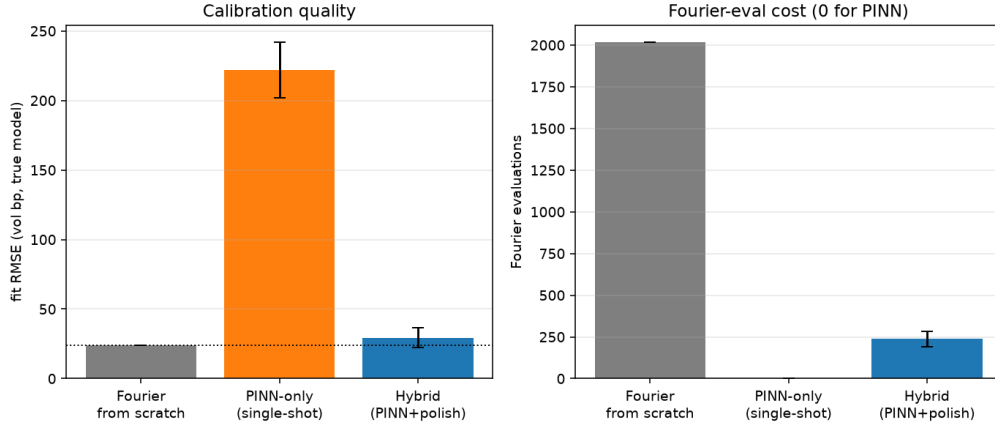


Figure 3: Hybrid calibration. Fit quality (left) and Fourier-evaluation cost (right) for the three methods on the BTC slice: the hybrid matches from-scratch quality at $\sim 9\times$ fewer Fourier evaluations, while PINN-only is fast but inaccurate.

exponential 50^{n+1} count is the *naive* full-tensor grid; sparse grids, ADI, and tensor-train methods reach higher dimensions, so the defensible claim is that a competitive finite-difference solve for the lifted rough-volatility PDE at $n = 16\text{--}32$ is impractical, not impossible. Even in this conservative form, the contrast — sub-linear error growth and bounded wall-clock against a grid that cannot be stored — is a concrete instance of the curse-of-dimensionality advantage that motivates mesh-free neural PDE solvers.

The solve/lift decomposition as a general lesson. A recurring hazard when a surrogate replaces an exact solver is to validate the surrogate against the wrong reference. The network solves the finite- n lifted PDE, whose exact solution is the lifted Monte Carlo, not the true ($n \rightarrow \infty$) model; comparing directly to the true model conflates the network’s solve-error with the model-approximation (lift) error. We found the two are of opposite sign and that the historical “PINN-versus-truth” gap was dominated by the lift, not the network. The remedy — validate against a same-fidelity reference and report the two errors separately — and the resulting crossover rule for the approximation order are not specific to rough volatility; they apply to any surrogate built on a controllable model reduction, and we would encourage their use whenever a learned solver sits on top of an approximated operator.

Calibration and the role of the parametric network. The applied study is honest about a negative result and its resolution. A single network spread over a five-parameter box is markedly less accurate than a point-trained one, worst in the rough, high-vol-of-vol corners where the PDE is stiffest, and calibrating with it alone is fast but unreliable — it fits its own imperfect prices, and the single-slice optimum can fall outside the training box. The parametric network nonetheless earns its keep as a differentiable warm start: seeding a short local optimization against the true pricer with its guess recovers from-scratch calibration quality at about an order of magnitude fewer true-model evaluations. The value proposition is therefore amortized — train once, then recalibrate cheaply — with the true pricer retained in the loop only for a brief, decisive polish.

Limitations. Our evidence rests on a single market snapshot (one Deribit BTC surface), and the parametric and calibration studies use a single maturity slice; timings are single-machine CPU

figures at moderate training budgets. The parametric network fixes V_0 and the maturity, its calibration fit to the wide crypto smile is coarse (496 vol bp at the surface level), and its solve-error carries a mild directional bias (a slight over-pricing of at-the-money and call strikes that grows with n). We study $n \leq 32$; the sub-linear trend need not persist arbitrarily far.

Future work. Several extensions are natural: adding the maturity as a network input to price whole surfaces rather than single slices; wider and deeper networks with GPU training to shrink the parametric corner penalty and to test the scaling to larger n ; transfer to other rough models (for example rough Bergomi) and to American or path-dependent payoffs; and a systematic study of the warm-start polish across market regimes. We would also like to characterize the high- n solve-bias and whether it can be removed by loss reweighting or a residual correction, and to benchmark the mesh-free approach directly against structured finite-difference schemes at moderate n where the latter remain feasible.

References

- E. Abi Jaber. Lifting the Heston model. *Quantitative Finance*, 19(12):1995–2013, 2019. doi: 10.1080/14697688.2019.1615113.
- E. Abi Jaber and O. El Euch. Multifactor approximation of rough volatility models. *SIAM Journal on Financial Mathematics*, 10(2):309–349, 2019. doi: 10.1137/18M1170236.
- C. Bayer, P. Friz, and J. Gatheral. Pricing under rough volatility. *Quantitative Finance*, 16(6): 887–904, 2016. doi: 10.1080/14697688.2015.1099717.
- C. Bayer, B. Horvath, A. Muguruza, B. Stemper, and M. Tomas. On deep calibration of (rough) stochastic volatility models, 2019. Preprint, arXiv:1908.08806.
- M. Bennedsen, A. Lunde, and M. S. Pakkanen. Decoupling the short- and long-term behavior of stochastic volatility. *Journal of Financial Econometrics*, 20(5):961–1006, 2022. doi: 10.1093/jffinec/nbaa049.
- P. Carr and D. B. Madan. Option valuation using the fast Fourier transform. *Journal of Computational Finance*, 2(4):61–73, 1999. doi: 10.21314/JCF.1999.043.
- F. Comte and E. Renault. Long memory in continuous-time stochastic volatility models. *Mathematical Finance*, 8(4):291–323, 1998. doi: 10.1111/1467-9965.00057.
- O. El Euch and M. Rosenbaum. The characteristic function of rough Heston models. *Mathematical Finance*, 29(1):3–38, 2019. doi: 10.1111/mafi.12173.
- O. El Euch, M. Fukasawa, and M. Rosenbaum. The microstructural foundations of leverage effect and rough volatility. *Finance and Stochastics*, 22(2):241–280, 2018. doi: 10.1007/s00780-018-0360-z.
- J. Gatheral, T. Jaisson, and M. Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6): 933–949, 2018. doi: 10.1080/14697688.2017.1393551.
- J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.

- S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993. doi: 10.1093/rfs/6.2.327.
- B. Horvath, A. Muguruza, and M. Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21(1):11–27, 2021. doi: 10.1080/14697688.2020.1817974.
- A. Jacquier and Ž. Zurič. Random neural networks for rough volatility, 2023. Preprint, arXiv:2305.01035.
- A. Kendall, Y. Gal, and R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7482–7491, 2018. doi: 10.1109/CVPR.2018.00781.
- A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 34:26548–26560, 2021.
- L. D. McClenny and U. M. Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023. doi: 10.1016/j.jcp.2022.111722.
- A. Papapantoleon and J. Rou. A time-stepping deep gradient flow method for option pricing in (rough) diffusion models. *Quantitative Finance*, 25(12):2009–2020, 2025. doi: 10.1080/14697688.2025.2572318.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- J. Sirignano and K. Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018. doi: 10.1016/j.jcp.2018.08.029.
- S. Wang, X. Yu, and P. Perdikaris. When and why PINNs fail to train: a neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.
- S. Wang, S. Sankaran, and P. Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024. doi: 10.1016/j.cma.2024.116813.